10. Poročilo: Spektralna analiza in filtriranje

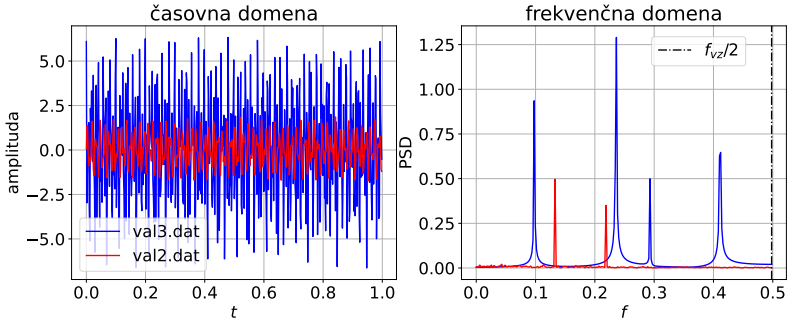
Modelska analiza I, 2023/24

Aljaž Ravnikar, dipl. inž. str. (UN), 23232128

7. februar 2024

1 Določevanje frekvenčnega spektra

Podana imamo dva signala, prikazana na spodnji sliki, poleg katerih je tudi spektralna gostota moči (PSD). Tam opazimo, da je frekvenčni spekter signala 2 ustrezen, pri signalu 3 pa je prišlo do puščanja; očitno dolžina vzorčenja ni enaka periodi signalov! Grafi prikazujejo le pozitivne frekvence, dobljene s hitro fourierovo transformacijo (FFT).



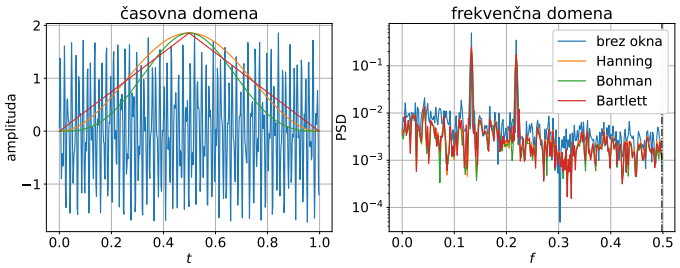
Slika 1.1: Surova signala

Poskusimo uporabiti oknenske funcije za zmanjšanje puščanja. Zaradi večje nazornosti puščanja, je frekvenčni prostor prikazan v log skali. Ponovno vidimo, da puščanja na signalu 2 sploh ni. Z njim tudi hitro obupam, saj z različnimi okni signalu ne pridem do živega (se ne izboljša), dočim pri signalu 3 prihaja do očitnih sprememb. Vrh je postal mnogo ožji.

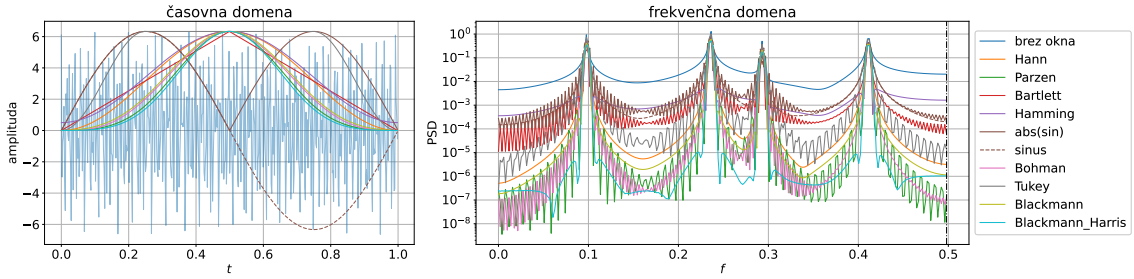
Opazim, da nekatera okna povzročajo oscilacije, taka v tem primeru niso uporabna. Najbolj izraziti vrhovi so pri oknu *Blackmann-Harris*, vendar je pri njem prisotnih nekaj oscilacij. Gladke prehode omogočajo tudi okna *Blackmann*, *Hann*, *sinus* in *Hamming*. Zanimivo je, da moje izmišljeno okno *sinus* omogoča gladke prehode vendar zajema tudi negativne vrednosti, dočim je izmišljeno okno *abs(sin)* vedno pozitivno vendar vsebuje oscilacije. Domnevam, da je vir oscilacij nezvezen prehod na sredini. Vendar velja poudariti, da sta bili ti dve izmišljeni okni uspešnejši od nekaterih uveljavljenih (ki jih nisem prikazoval na grafu).

Opomba: Še najbolje se je obnesla vizualna ocena frekvenčne domene. Apliciranje metod, ki temeljijo na razliki vrh-dolina je pri oscilirajočih primerih lahko težavno. Okarakterizacija puščnja je v splošnem težka in specifična za vsak signal.

Poizkusil sem tudi s krajšanjem signala, tako na koncu, kot tudi periodično. Opazimo, da krajšanje signala na koncu predvsem veča puščanje na signalu 3 (nisem uporabljal okenskih funkcij, vendar z njimi pričakujem podoben rezultat). Pri dovolj kratkem signalu 2 je prišlo tudi do popačenja, očitno sem zgrešil periodo signala. Če pa periodično izpuščamo podatke z izbranim korakom (tako torej višamo frekvenco vzorčenja), že zelo hitro pride do potujitve in PSD postane neuporaben.

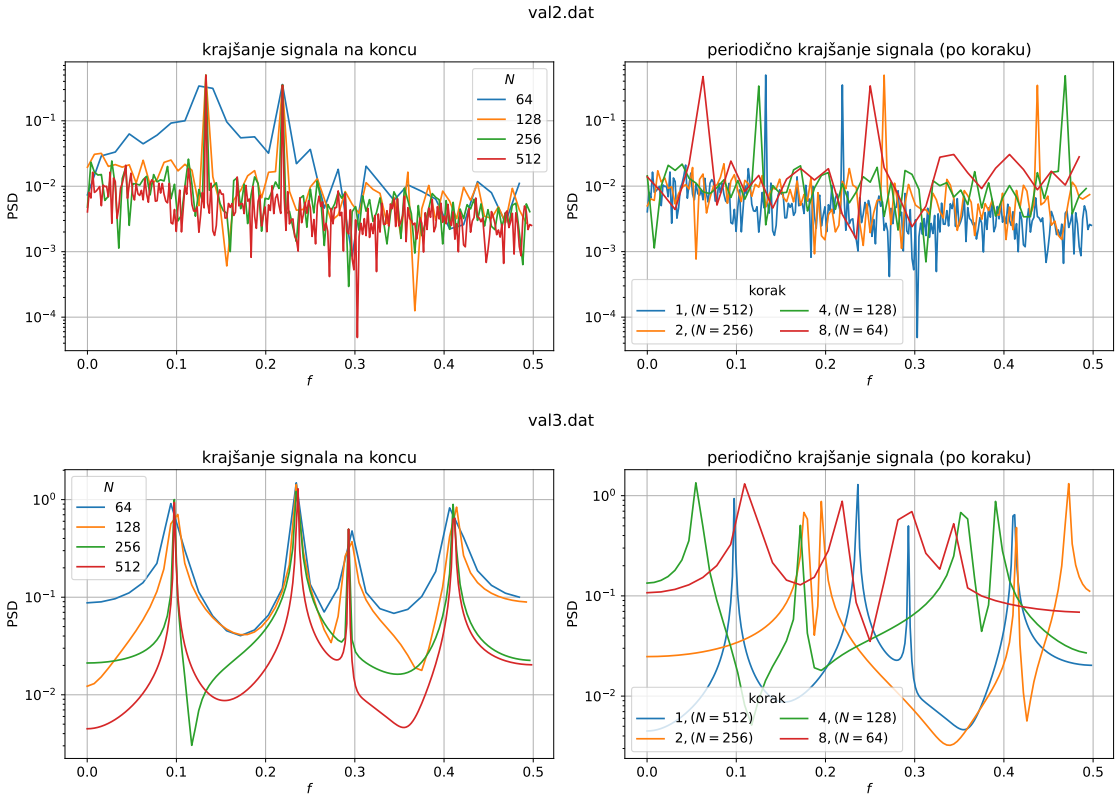


(a) val2.dat



(b) val3.dat

Slika 1.2: Analiza signalov z okenskimi funkcijami



Slika 1.3: Izpuščanje podatkov brez okna

2 Dekonvolucija signalov z Wienerjevim filtrom

Dekonvolucija signala $u(t)$ se lahko izvede spektralno s Fourierovo transformacijo (diskretno $\mathcal{F}_N$), če je izmerjen konvoluiran signal $c(t)$ s prenosno funkcijo $r(t)$ podan kot $c(t) = (u * r)(t)$:

$$u(t) = \mathcal{F}_N^{-1}\left[\frac{\mathcal{F}_N(c)}{\mathcal{F}_N(r)}\right]. \tag{2.1}$$

Ker je dekonvolucija zelo občutlova na šum, uporabimo filtriranje. Tukaj se osredotočam na Wienerjev filter, ki predpostavlja, da izmerjen konvoluiran signal lahko razstavimo na dve komponenti

$$c(t) = s(t) + n(t), \quad s(t) = (u * r)(t), \tag{2.2}$$

kjer je $n(t)$ šum in $s(t)$ signal brez šuma, ki ga želimo izluščiti.

Tako je Wienerjev filter

$$\Phi(f) = \frac{|S(f)|^2}{|S(f)|^2 + |N(f)|^2} = \frac{|C(f)|^2 - |N(f)|^2}{|C(f)|^2} \tag{2.3}$$

s katerim pomnožimo zašumljen $\mathcal{F}_N$ izmerka pred dekonvolucijo

$$\tilde{u}(t) = \mathcal{F}_N^{-1}\left[\Phi(f)\,\frac{C(f)}{R(f)}\right], \tag{2.4}$$

pri tem so S, N, C in R Fourierove transformiranke s, n, c. in r ter $\tilde{u}$ aproksimacija vhodnega signala.

dani so nam signali `signal{0,1,2,3}.dat`, kjer je 0 brez šuma, ostali imajo vedno večjo zašumljenost. V predlogi je podana tudi prenosna funkcija

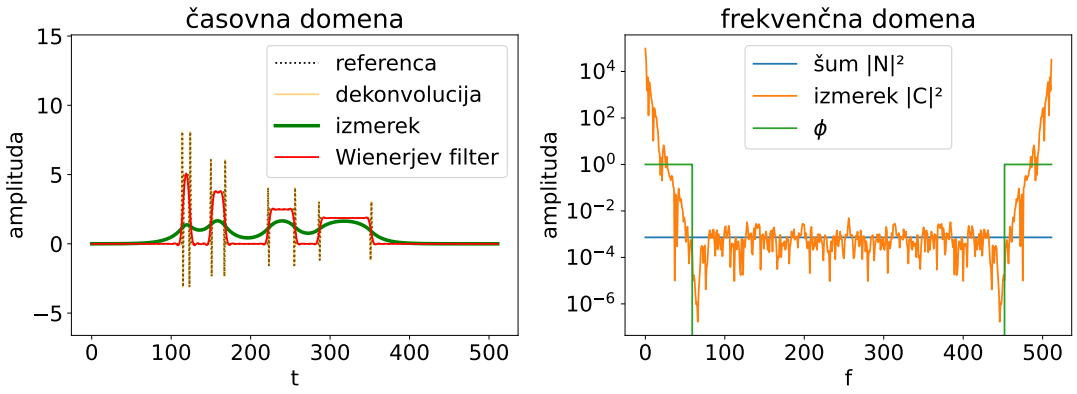
$$r(t) = \frac{1}{32}\exp\left(-\frac{|t|}{16}\right) \tag{2.5}$$

Wienerjev filter je potrebno določiti ročno. V tej nalogi sem uporabljal zelo preprosto konstrukcijo Wienerjevega filtra – določil sem mejo med področjem, kjer prevladuje šum in kjer signal ter šum utežil z nič in signal z 1. Navdušen sem bil nad tem, da tako preprost filter rešuje problem že tako dobro, da sploh nisem razmišlja o boljšem. Iz primerjave s kolegi, ki so imeli bolj sofisticirane filtre tudi na oko ni zaznati izrazitega izboljšanja. Implementacija filtrov iz knjižnice `scipy` in `skimage` se ni izkazala za uspešno.

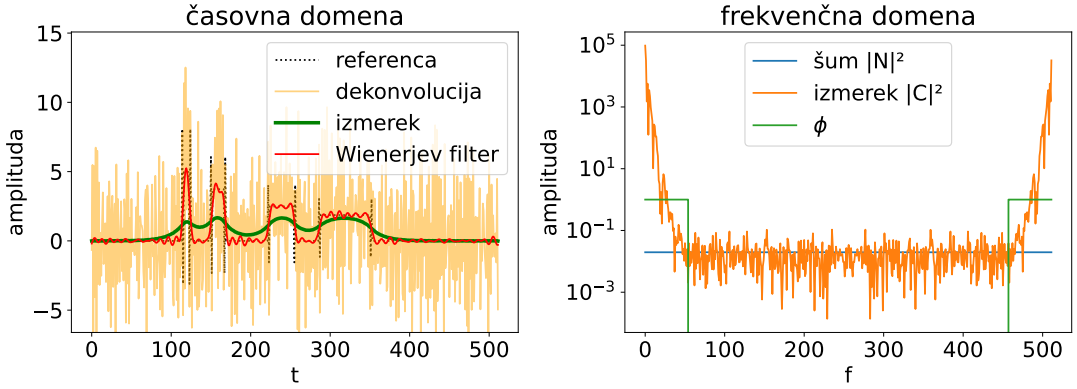
Ker je signal0 brez šuma, je rezultat dekonvolucije že dokaj lep in potrebe po filtru ni. Vendar tudi aplikacija filtra ne pokvari rezultata in je dekonvolucija z njim tudi zelo dobra. V sledečih primerih bom podajal rezultat dekonvolucije brez šuma kot referenco!

V signalu1 je že nekaj šuma, vendar le-ta v časovni domeni ni opazen, ampak vpliva na dekonvolucijo, ki vsebuje že toliko šuma, da je neberljiva. Z malo domišljije lahko sicer izluščimo skočne funkcije. Wienerjev filter pa to težavo odpravi in dobimo manj zašumljen signal, podoben onemu iz signala0, le malenkost več je oscilacij.

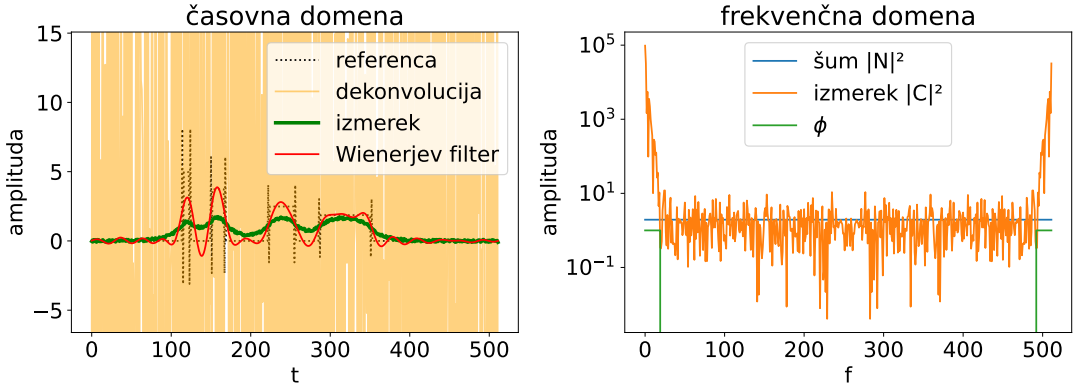
Signal2 vsebuje že opazen šum, signal3 pa očitnega. Pri obeh direktna dekonvolucija odpove, wienerjev fiter pa do neke mere rešuje zadevo, saj je v tem primeru precej (še več) oscilacij.



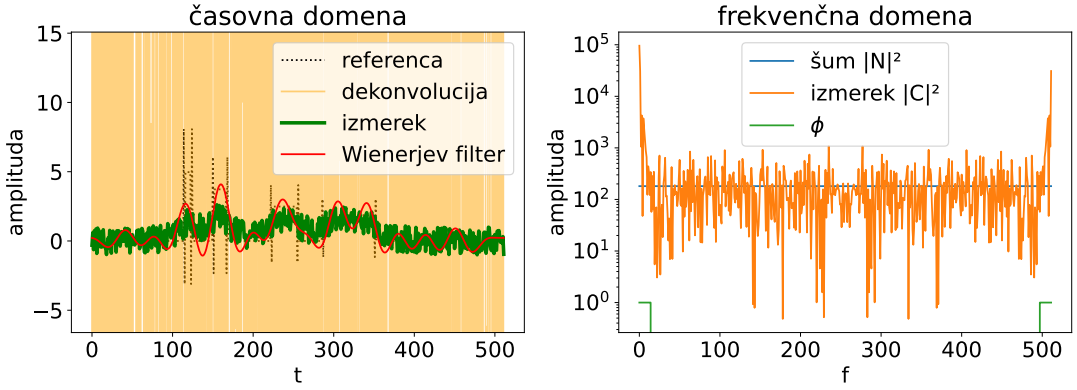
(a) `signal0.dat`



(b) `signal1.dat`



(c) `signal2.dat`

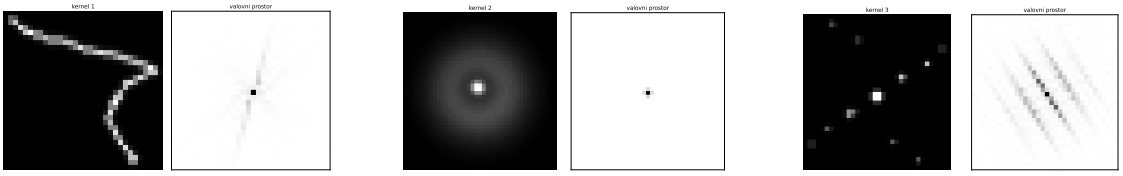


(d) `signal3.dat`

Slika 2.1: Dekonvolucija signalov z Wienerjevim filtrom

3 Rekonstrukcija fotografije

Podane so slike Lene, razmazane s različnimi konvolucijskimi jedri/kernel (na sliki spodaj) in različnimi količinami Gaussovskega šuma.



(a) Jedro 1: tresoč objektiv (b) Jedro 2: slab fokus (c) Jedro 3: ukl. mrežica

Slika 3.1: Konvolucijska jedra

Problem dekonvolucije se rešuje na podoben princip kot v prejšnjem poglavju, le da tokrat uporabljamo FFT v dveh dimenzijah. Kot prvo sem poizkusil rekonstruirati kernel 1 brez šuma in kmalu ugotovil, da dekonvolucijo zmoti rob slike, zato sem uporabil različne okenske funkcije, kar pa močno zatemni sliko, čemur sem dodal različno število pikslov (px) po robu na načine, predstavljene na spodnji sliki. Opomba: dodatek px konstantne vrednosti je ekvivalenten nerazširjeni sliki; zaradi narave kode, brez dodatka ni šlo.



Slika 3.2: Dekonvolucija slike `lena_k1_n0.pgm` z različnimi okni in razširitvami

Ugotovil sem, da dodatek 200 px sliko močno izboljša, skupaj z uporabo primerne okenske funkcije (okna). Izgleda da se najbolje obnese razširitev *linear ramp*, vendar razlike vizualno niso tako očitne pri uporabi okenskih funkcij (opomba: *boxcar* je naziv za okno konstantne vrednosti 1, torej kot bi okenske funkcije ne bilo!). Nekatera okna zelo zatemnijo sliko, zato tu niso prikazana. Vizualno lahko sklepam, da je dodatek 200 px razširitve linear ramp in okna triang (Bartlett), Hann, Barthann najugodnejše.

V modulu `skimage.restoration` sem našel implementacijo algoritma *Richardso-Lucy* (LR), ki ob znani PSI (point spread function), kar je naš kernel, poskuša iterativno povrniti (dekonvoluirati) originalno sliko. Kot je razvidno iz spodnje slike, je pri tem algoritem precej uspešen, sploh če mu pomagamo z razširitvijo slike, kot pri klasični dekonvoluciji. Pri nerazširjeni sliki so artefakti na robu lepo vidni, pri prevelikem številu iteracij pa prihaja do popačenja, ki se kot valovi širi preko celotne slike, če še dodatno večamo število iteracij. To se venomer zgodi v spodnjem levem robu in izgleda, kot da bi se algoritem "ujel v Lenine lase". Izkaže se, da nekako 50 iteracij vrne zadovoljiv rezultat, ki se nadaljno ne izboljšuje pretirano. Potrebno je poudariti, da je ta algoritem tudi bolj računsko zahteven od naše spektralne metode.



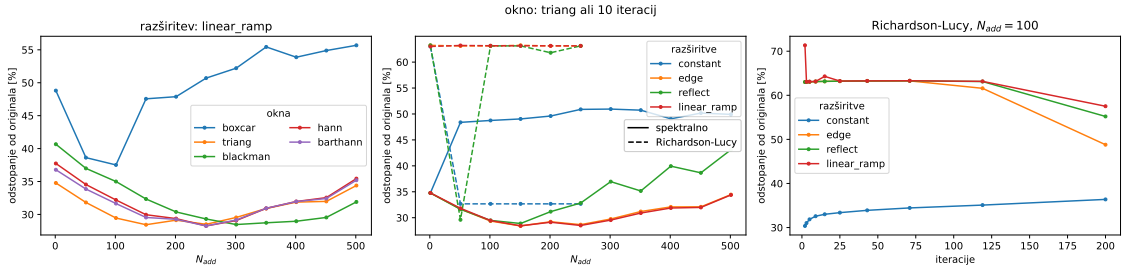
Slika 3.3: Dekonvolucija slike `lena_k1_n0.pgm` z različnimi razširitvami in iteracijami

Da lahko bolje ovrednotim rezultate, sem uporabil originalno sliko in jo pretvoril v grayscale zapis, s katerim sem primerjal rezultate dekonvolucije na sledeč način:

$$\eta = \frac{||D_{ij} - R_{ij}||}{||S_{ij} - R_{ij}||}, \tag{3.1}$$

kjer so R, D in S matrike referenčne(originalne), dekonvoluirane in signalne(podane) slike. Torej, če z dekonvolucijo slike nismo čisto nič izboljšali, je vrednost relativne napake η obdelane slike 100%, če smo pa uspeli popolnoma rekonstruirati original, pa ima η vrednost nič.

Za primere, prikazane v zgornjih dveh slikah lahko izračunamo vrednosti relativnega odstopanje od originala v odvisnosti od števila dodanih px (N_{add}). Ugotovim podobno, kot sem sklepal vizulano; najhitreje pade napaka pri oknu triang, vendar se kasneje razlike med okni zmanjšajo. Edino brez okna (boxcar) se obnaša nepredvidljivo, a ne da uporabnih rezultatov. Razlike med tipi razširitev niso tako velike pri manjših razširitvah, linear-ramp in endge sta celo identični (velja le za okno triang!), dočim je rezultat za algoritem LR presenetljiv. Tega v tem trenutku ne znam pojasniti. Prav tako sem pričakoval, da bo pri iteracijah nad 100 prišlo do poslabšanja, vendar je začuda temu ravno obratno. Algoritem LR sicer vrne malenkost manj ostro sliko, vendar bolj kontrastno od dekonvolucije.
Opombi: Število iteracij pri LR se na grafu prične z 1 zaradi tehničnih razlogov. Pri opazovanju napak datotek `lena_k2_n0.pgm` in `lena_k3_n0.pgm` je potek drugačen (predvsem za algoritem LR)!



Slika 3.4: Napaka dobljene slike glede na pravo Leno za datoteko `lena_k1_n0.pgm`

Najbolj perspektivno za obravnavo se izkaže okno triang in razširitev linear-ramp med 150 in 250 px. Omeniti velja, da manjša razširitev pohitri izračun, prav tako pa je potrebnih manj iteracij LR. Do sedaj sem obravnaval le datoteko `lena_k1_n0.pgm` in ugotovil, kaj se dobro obnese, sedaj po bom te metode poskušal aplicirati tudi na preostalih dveh slikah brez šuma.

Za lažjo nadaljno obravnavo lahko določimo nov parameter ujemanja rekonstruirane slike z originalom

$$1 - \eta. \tag{3.2}$$

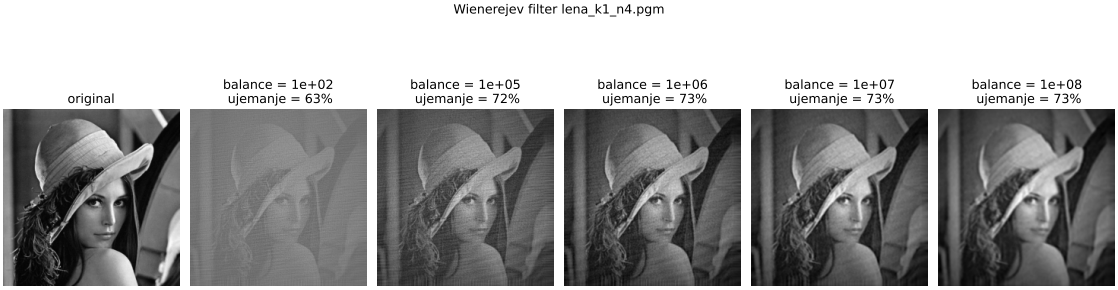
Tukaj η izračunamo po enačbi 3.1. Vrednost 1 pomeni popolno ujemanje z originalom, 1 pa brez izboljšave.

Sedaj lahko napademo še preostali dve sliki brez šuma, kjer uporabimo okno triang, razširitev linear-ramp z 200 px in 50 iteracij. Domnevam, da je zaradi narave jedra 2 (tresoč objektiv), ki spominja na Gaussovski šum, direktna dekonvolucija neuspešna, na jedru 3 pa deluje uspešno. Algoritem LR daje primerljive rezultate na jedru 1, navkljub (ponovno čudno) nizkim izračunom ujemanja. Na jedru 2 se obnese odlično v primerjavi z dekonvolucijo. Potrebno je opomniti, da vrne algoritem LR na jedru 3 boljše rezultate pri manj iteracijah, nekako med 10 in 20, kar je uporabljeno v prilogi, vendar rezultati LR vseeno niso primerljivi z dekonvolucijo.



Slika 3.5: Dekonvolucija preostalih slik brez šuma z najboljšimi okni in razširitvami

Za analizo slik z dodanim Gaussovskim šumom moramo ponovno uporabiti filter. Naš delovni konj je v tem primeru Wienerjev filter. Uporabljam filter z Wiener-Hunt-ovim pristopom iz knjižnice `skimage.restoration`, ki je spisan za n-dimenzij. S parametrom *balance* omogoča, da krmilimo med obnovitvijo podatkov in artefaktom zaradi šuma. Na primeru jedra 1 in RMS šuma 4 je to prikazano na spodnji sliki. Opazimo, da se slika vidno izboljšuje do nekje $10^6 - 10^7$, dalje se sicer ujemanje veča, vendar slike opazno izgublja ostrino, zato na tem mestu lahko dvomim o ustreznosti parametra ujemanja!



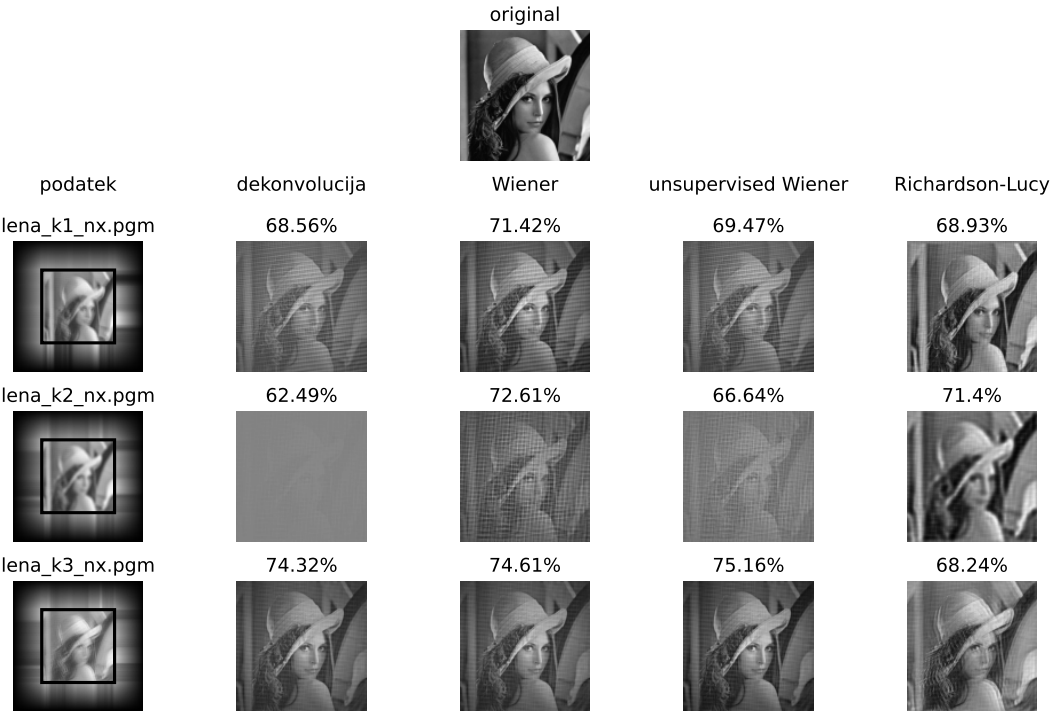
Slika 3.6: Dekonvolucija zašumnljene slike `lena_k1_n4.pgm` z različnim prametrom *balance* v Wienerejevem filtru

Na tem mestu sem ugotovil, kako lahko umerim filter za kar najboljši rezultat, kar je predstavljeno v spodnji preglednici. Kot sem omenil že poprej, jedro 2 v splošnem rabi malenkost večji parameter, zanimivo je pri jedru 3, vendar je rezultat lep in boljši od konkurenčnih metod (Richardson-Lucy in unsupervised_Wiener iz knjižnice `skimage.restoration`). Slednji iterativno in stohastično oceni parametre dekonvolucije, vendar je v splošnem slabši (in počasnejši) in Wienerja, če poznamo oceno šuma zanj. Tako dekonvoluirane slike v končni verziji so prikazane v prilogi.

kernel\šum(RMS)	0	4	8	16
1	10^6	10^6	10^7	10^7
2	10^6	10^7	10^8	10^8
3	10^0	10^6	10^6	10^6

Tabela 3.1: Vrednosti parametra *balance* za kar najboljšo filtracijo

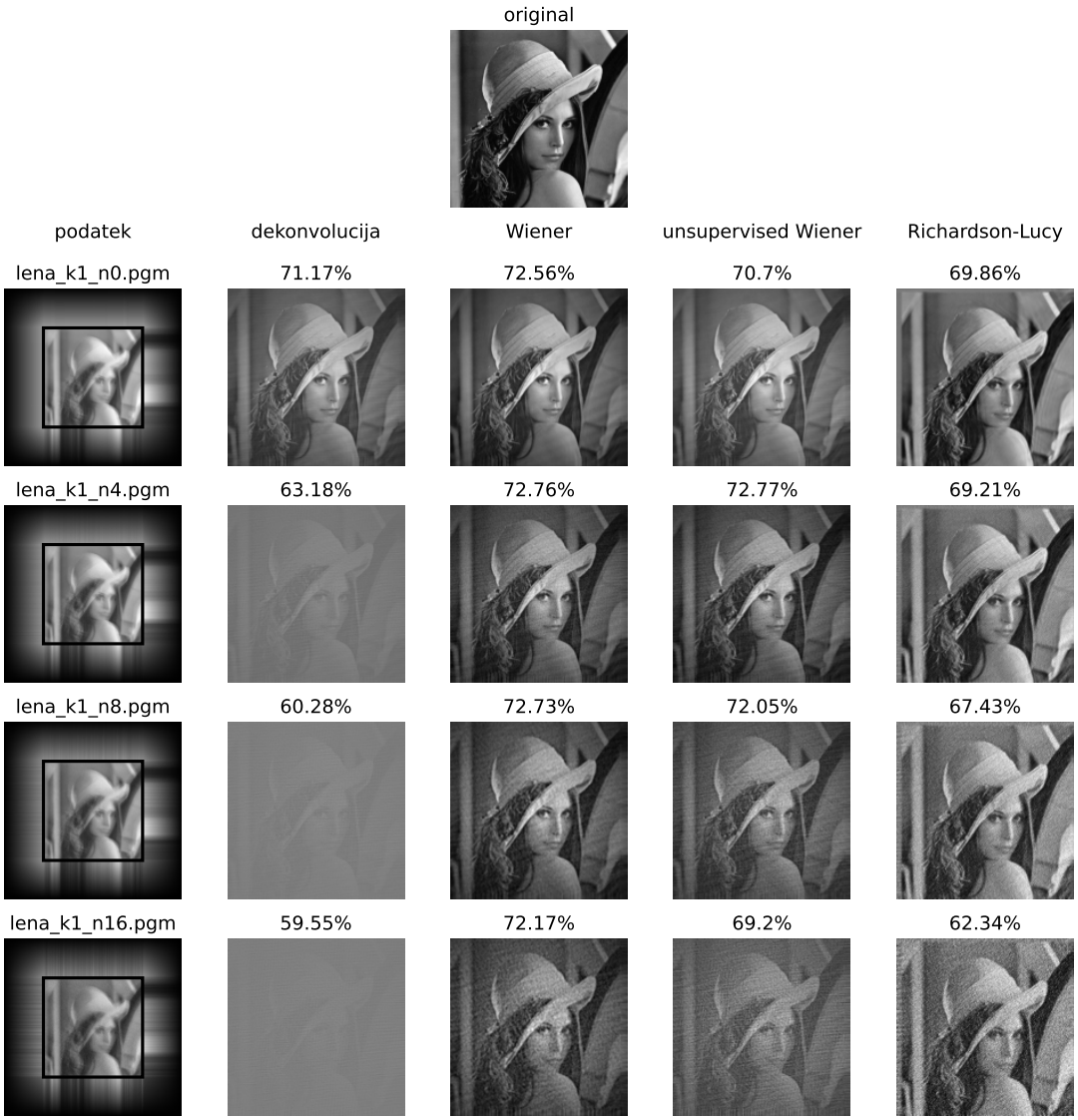
Periodično motnjo sem napadel s taistimi metodami kot Gaussovski šum, vendar Wiener ni več tako uspešen, saj se periodične motnje pojavljajo tudi na dekonvoluirani sliki le še bolj izrazite. Presenečen pa sem, da se algoritem LR odziva enako kot na slike brez periodične motnje. Tudi ujemanja so podobna onim brez periodične motnje, čeprav manjša od Wienerjevih, vendar je slika vizualno lepša. Zanimivo, da je tudi izračun napake pri jedru 1 in algoritmu LR v tem primeru pravilno izračunan, takisto tudi v prilogi. Napake v kodi, ki bi bila kriva za anomalije na slikah 3.4 in 3.5 nisem našel!



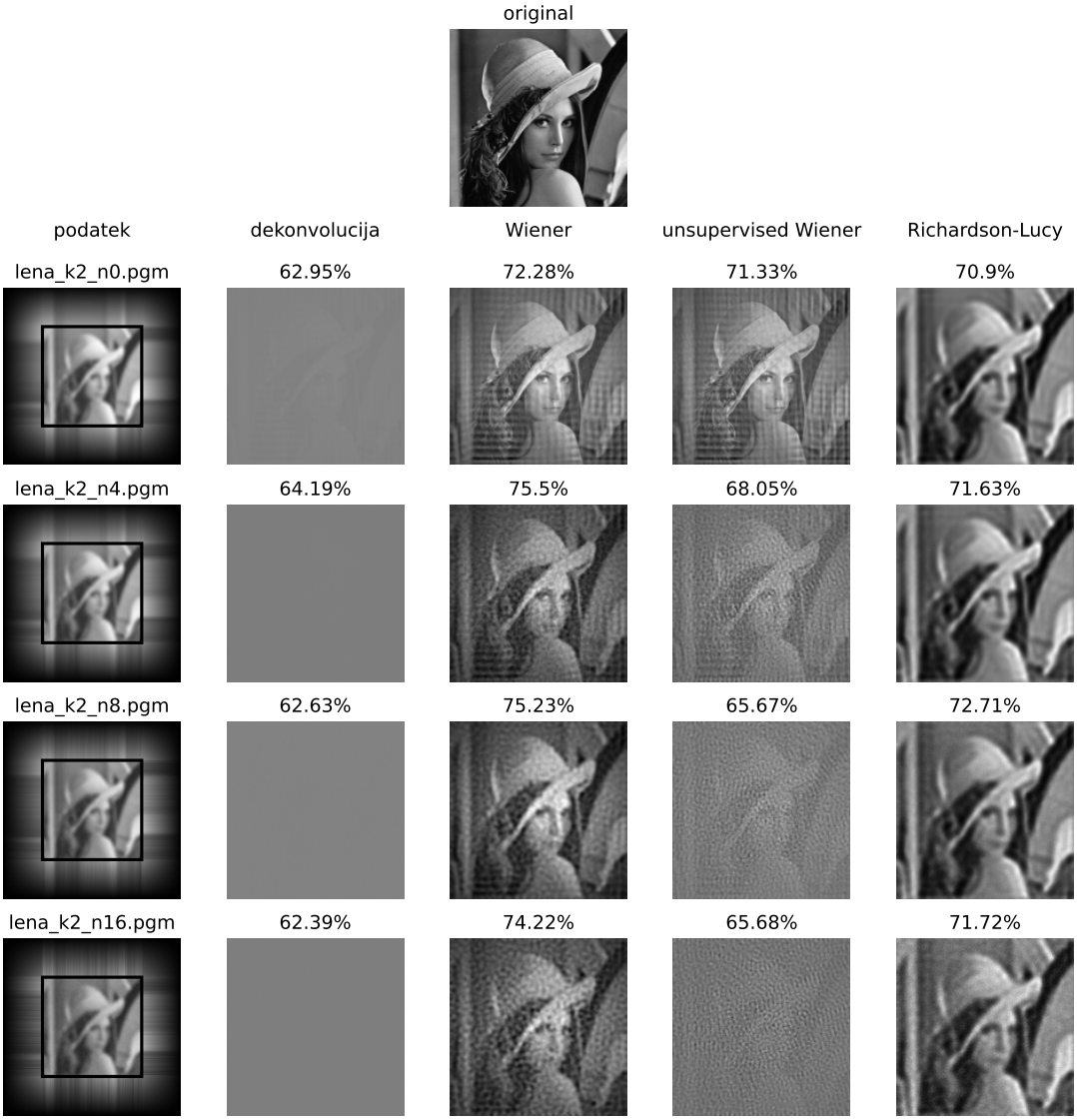
Slika 3.7: Poskus dekonvolucije slik s periodično motnjo

4 Priloga: končni rezultati dekonvolucije Lene

V vseh primerih je uporabljenih 200 px razširitve z linear_ramp in okenska funkcija triang. Število iteracij algoritma Richardson-Lucy je 50, razen za jedro 3, kjer jih je 10. Parameter balance je tak, kot je ugotovljeno v tabeli 3.1.
Pomembna opomba: Izračunane parametre ujemanja je potrebno jemati z rezervo, še vedno priporočam vizualno primerjavo!



Slika 4.1: Kernel1



Slika 4.2: Kernel2



Slika 4.3: Kernel3